

ContribAI

AI Day Noida · 20 September 2026 · Paytm Office, Noida Track: Problem Statement 04 — Open Innovation: Build What Matters with AI Domain: AI developer tools

An agent that reads an open-source issue, grounds it in the repository's actual files and health signals, and returns an executable contribution plan — not a summary. Every score shows its work, and the product still works when the model is unreachable.

1. Problem statement and target user

Target user. A competent developer who has never landed a pull request on a project they do not own. Roughly: two or more years of writing code, comfortable in one or two languages, zero merged contributions to someone else's repository.

The pain point. The blocker is not motivation and it is not a shortage of open issues. It is the gap between "this issue is labelled good first issue" and "this issue is a realistic first pull request for me, and here is what it would take."

Closing that gap by hand, per issue, costs 30–60 minutes of unstructured reading: open the issue, skim the thread, guess which files are involved, check whether the project is still alive, check whether a maintainer will ever review it, try to estimate whether you personally can finish it. Most people run that loop three or four times, conclude they are underqualified, and stop. The abandonment happens *before* any code is written.

Existing alternatives and the gap.

Alternative	What it does	What it does not do
GitHub "good first issue" label	Filters by a maintainer's one-time guess	Knows nothing about <i>you</i> ; the label is often stale or wrong
goodfirstissue.dev, Up-For-Grabs	Aggregate labelled issues	Still a flat list; no per-contributor fit, no plan
GitHub repository recommendations	Suggests <i>repositories</i> from your stars	One level too coarse — the unit of contribution is an issue
Asking an LLM directly	Summarises an issue you paste in	No repository grounding — invents file paths and build commands; no ranking across candidates

The gap is one level below every existing tool: **given this specific issue and this specific contributor, is this a realistic contribution, and what are the concrete steps?** That requires modelling both sides and grounding the answer in the repository — a question no existing tool in the table above actually answers.

2. Why Open Innovation, and how the track's requirements are met

Open Innovation asks teams to find a real problem worth solving rather than answer a prescribed one, and to show a clear line from that problem to the AI capability used. The problem here was not chosen to fit a technology. It was chosen because the drop-off is measurable, the affected population is large, and every existing tool stops one level above where the decision actually gets made.

PS04 core requirement	Where it is answered
Clearly define the target user and pain point	§1 — a developer with 2+ years of code and zero merged external PRs; the 30–60 minute per-issue evaluation loop that ends in abandonment before any code is written
Explain existing alternatives and the gap being addressed	§1 — four named alternatives (GitHub labels, aggregators, repo recommendations, raw LLM use) and the specific gap each leaves
Demonstrate a working core user flow	§3 — seven steps, sign-in to contribution plan, with a verified end-to-end run captured from a live server
Use AI where it provides a meaningful advantage	§5 — including the deliberate decision about where <i>not</i> to use it
Define at least one measurable success or impact metric	§8 — recommendation-to-first-PR conversion, plus four falsifiable supporting metrics
Explain data requirements, model/API dependencies and limitations	§6 (data and dependencies) and §9 (seven stated limitations)
Describe how the prototype could be scaled or deployed	§10 — stateless API, out-of-process ingestion, per-issue analysis cache amortised across all users

Domain and advanced directions: AI developer tools, plus agentic automation and intelligent workflows — tool calling against the GitHub API, retrieval and semantic search, structured extraction and classification, and multi-step task planning. The agent plans; it never files, comments or claims on the user's behalf.

3. Core user flow (the demo)

1. **Sign in.** Pick a seeded contributor profile, or sign in with GitHub to analyse a real account.
2. **Skill graph is built from evidence.** Repository languages, dependency manifests, commit recency and merged pull requests — each skill shows the evidence that produced it and a confidence between 0 and 1. Nothing is self-reported unless the user explicitly adds it.
3. **Issues are ranked.** The corpus is filtered, retrieved and scored. The UI reports the funnel honestly: "23 issues analysed · 18 passed hard filters · 11 strong matches", plus why each rejected issue was dropped.
4. **Open a recommendation.** Fit score with all eight dimensions exposed as bars, the deterministic reasoning list, matched skills, and skill gaps with hour estimates.

5. **Understand the issue.** Summary, problem description, why it matters, difficulty with its basis, effort range, concepts, likely files, investigation order — and an explicit "*what the agent does not know*" panel.
6. **Get the plan.** 8–11 executable steps from reading the contribution guide through opening the pull request.
7. **Ask follow-ups.** Grounded chat over that one issue and repository.

Verified end-to-end run (alex, developer_match mode, no API key configured):

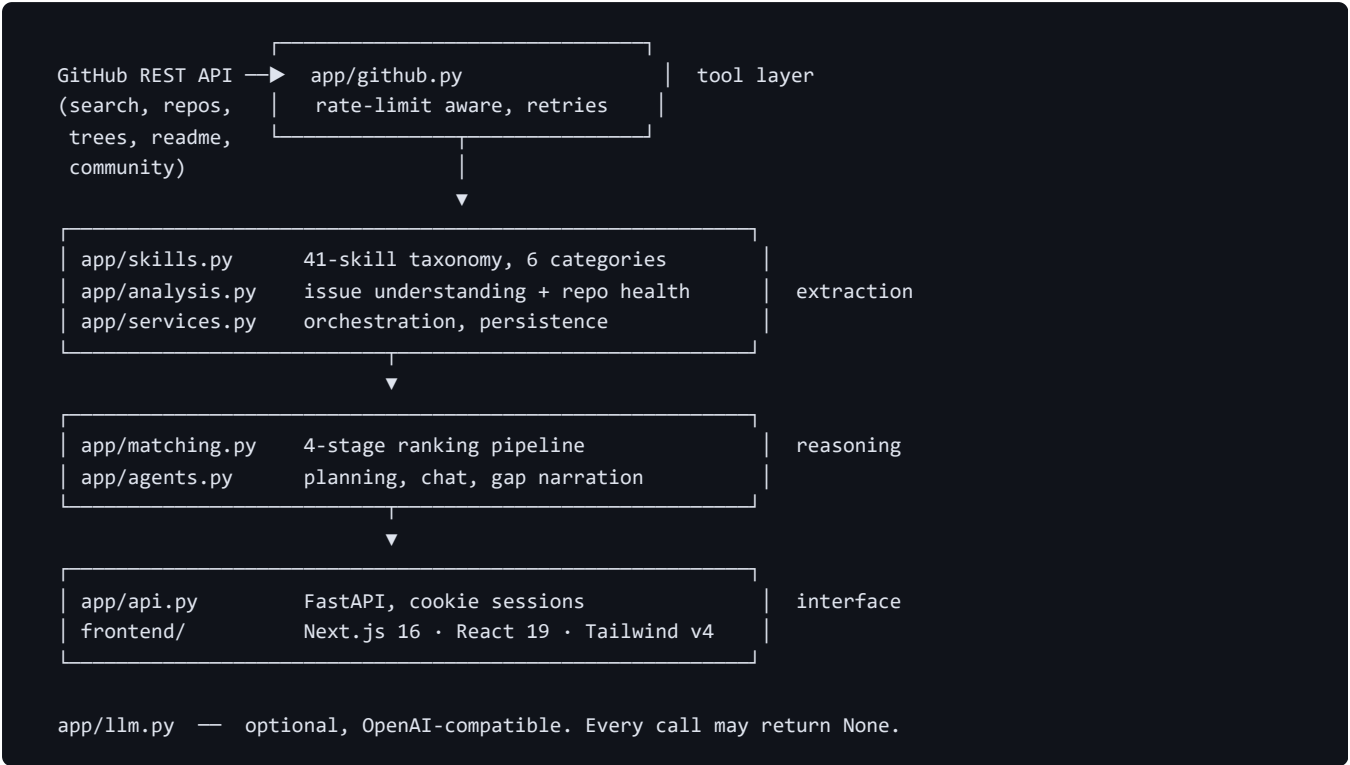
```
stats: 23 analysed · 18 passed filters · 11 strong matches
dropped: 1 security-sensitive, 4 outside selected technologies

0.91 best_fit ledgerly/ledgerly-api#1863 CSV import silently drops rows with a malformed date
0.90 best_fit ledgerly/ledgerly-api#1842 Pagination returns wrong page offsets when a transaction filter is a

plan source: heuristic
  3. Fork and clone           gh repo fork ledgerly/ledgerly-api --clone
  4. Install dependencies    uv sync
  5. Run the existing test suite  pytest -q
  7. Read src/ledgerly/services/import_service.py
  8. Create a branch         git checkout -b fix/issue-1863
 10. Add a regression test
 11. Run the full suite and open the PR    pytest -q
```

uv sync and pytest -q are not generic guesses — they were read from that repository's actual setup commands, and import_service.py from its actual file tree. This is the grounding rule working: **no invented build systems**.

4. Architecture and technology overview



Layer	Choice	Why
API	FastAPI + SQLAlchemy 2.0	Typed schemas double as the API contract
Database	SQLite by default, Postgres via <code>DATABASE_URL</code>	Demo runs with zero infrastructure
Model	Groq <code>openai/gpt-oss-120b</code> over the OpenAI-compatible wire format	One <code>httpx</code> POST, no vendor SDK. Swapping to Together, OpenRouter or local vLLM is a base-URL change
Retrieval	TF-IDF cosine, ~40 lines, in-process	At a few hundred candidate issues this is exact, instant, offline and debuggable. Embeddings when lexical overlap measurably misses — not before
Auth	GitHub OAuth, <code>read:user</code> scope only; tokens Fernet-encrypted at rest	Minimum scope. The token is never serialised into any response schema
Frontend	Next.js 16, React 19, Tailwind v4, Phosphor icons	Local components, no component framework and no state library

Roughly 4,300 lines of application code and 43 passing tests.

5. The AI component, explained

This is the part a jury should read closely, because the interesting decision is **how little** the model is trusted.

Where AI earns its place — and where it does not

The problem has three distinct sub-problems, and only two of them are AI-shaped. Being explicit about which is which is the core design claim of this project.

Sub-problem	Approach	Why
<i>Is this issue disqualifying?</i> (closed, assigned, already has a PR, abandoned repo)	Rules. No model.	These are facts with known answers. A model here would add latency, cost and a failure mode in exchange for nothing.
<i>How hard is this issue, what does it need, which files does it touch?</i>	AI-shaped: extraction over unstructured prose.	An issue body is free text written by a stranger. Turning it into difficulty, required skills, concepts and candidate files is exactly what nothing but language understanding does well. Regex cannot read "the offset is computed before the filter is applied" and conclude it needs SQLAlchemy query knowledge.
<i>Given this issue and this person, what should they actually do?</i>	AI-shaped: synthesis across two unstructured sources.	Joining a contributor's evidence history against a specific repository's conventions to produce ordered, executable steps is judgement, not lookup.
<i>Which of 23 candidates ranks highest?</i>	Rules again — eight weighted dimensions.	The user has to be able to disagree with the ranking. A score that cannot explain itself is one a first-time contributor will not trust, and trust is the thing being rebuilt here.

So the AI is confined to reading prose and writing prose — the two places it is genuinely better than code — and kept out of the ranking and the filtering, where determinism is worth more than fluency. **Roughly 90% of the final score is computed without any model call.** That is not a limitation of the prototype; it is the design.

The advantage is concrete and measurable: a developer evaluating an issue by hand spends 30–60 minutes and usually quits. This returns a ranked shortlist with reasoning in under a second, and an executable plan on demand.

The four-stage pipeline

```
Stage 1  Hard filters          deterministic, disqualifying
        closed · assigned · PR already open · wontfix/duplicate/stale ·
        security-sensitive · repository inactive · single-maintainer with no guide

Stage 2  Semantic retrieval TF-IDF cosine between the contributor's profile vector
        and each issue document (title, body, labels, repo description, topics,
        extracted skills and concepts)

Stage 3  Structured scoring 8 weighted dimensions, asserted to sum to 1.0 at import
        skill_match .30 · difficulty_match .20 · technology_match .15 ·
        repository_accessibility .10 · issue_clarity .10 · effort_fit .05 ·
        learning_value .05 · repository_activity .05

Stage 4  LLM reasoning        prose only, layered on top of stages 1-3
```

Stage 3 is the product. The ranking is fully reproducible without a single model call, which is precisely why every recommendation can show its work. Semantic similarity is capped at 10% of the final score — it nudges ordering *within* a tier and never drives it.

What the LLM does and does not do

The model may	The model may not
Rewrite the issue summary in plain language	Be the source of any GitHub fact
Explain why the issue matters	Invent file paths, function names, APIs or maintainer opinions
Adjust difficulty within the allowed enum	Change <code>required_skills</code> or <code>affected_files</code> — these feed scoring and must stay reproducible
Propose investigation order and open questions	Invent a build system; unknown setup commands must leave <code>command</code> empty
Write the contribution plan prose	Write the patch — the contributor does the engineering

The fallback contract

Two rules the entire codebase depends on, stated at the top of `app/llm.py` :

1. **Every model call can return `None`** . Every caller has a deterministic fallback. There is no code path where an unreachable model breaks the product.
2. **The LLM only interprets data already gathered.** It is never the source of a fact.

The consequence is demonstrable: **the entire flow above — ranking, analysis, plan, chat — was verified with no API key configured.** `llm_available: false` . The output degrades in prose quality and is labelled `Deterministic` in the UI; nothing breaks. For a live demo on venue wifi, this is the difference between a product and a liability.

Provenance in the interface

Every AI-touched surface is labelled: `Deterministic` or `LLM-enriched` , plus `low / medium / high` confidence. Facts and inferences never share a line — the analysis panel separates "*REPOSITORY (fact)*" from "*ANALYSIS (inference, may be wrong)*", and that separation is enforced in the prompt construction itself, not just the display.

6. Data requirements, dependencies and demonstration dataset

What data the system needs

Data	Source	Required?	If unavailable
Issue title, body, labels, comments, assignee	GitHub REST	Yes — this is the object being reasoned about	Nothing to analyse
Repository language, topics, stars, last commit	GitHub REST	Yes — drives filters and difficulty	Repo scores as <code>unknown</code> activity, accessibility drops
Repository file tree	GitHub git/trees	No	Affected-file inference and setup-command detection are skipped; the planner leaves <code>command</code> empty rather than guessing
README, CONTRIBUTING, code of conduct	GitHub community profile	No	Accessibility score drops; the plan explicitly says no contribution guide exists
Contributor repos, manifests, merged PRs	GitHub REST (<code>read:user</code>)	No	Falls back to self-reported skills entered at onboarding

No training data is required. Nothing here is a trained model — there is a rule system, a TF-IDF index built at request time, and prompts against a general-purpose LLM. There is no dataset to collect, label or maintain before the system works, which is why a prototype can be honest on day one.

No personal data beyond a public GitHub profile is read, and the OAuth scope is `read:user` only — no repository access, no write permissions. Tokens are Fernet-encrypted at rest and never serialised into any response schema.

Model and API dependencies

Dependency	Role	Failure behaviour
GitHub REST API	Every fact in the system	Rate limit surfaces to the UI as a countdown, never a silent failure; the seeded corpus works offline
Groq <code>openai/gpt-oss-120b</code>	Prose: summaries, plans, chat	Optional. Every call may return <code>None</code> ; every caller has a deterministic fallback
OpenAI-compatible wire format	Provider portability	Swap to Together, OpenRouter, Anthropic-compatible gateways or local vLLM by changing one base URL — no SDK lock-in
SQLite / Postgres	Persistence	Connection-string change, no code change

The single most important property: **there is no hard dependency on any model vendor.** The product degrades in prose quality without one, and is labelled as such in the interface. It does not break.

Demonstration dataset

A seeded corpus ships with the repository so the demo needs no network, no GitHub token, and no API key.

- **5 repositories** spanning Python, TypeScript and Go, with deliberately varied health: `vela-ui/vela` (12.8k stars, health 1.00), `orbitcache/orbit` (8.2k, 0.87), `ledgerly/ledgerly-api` (3.4k, 1.00), `pipeforge/pipeforge` (2.2k, 0.87), `docsmith/docsmith` (640, 0.96) — including one deliberately quiet repository so the activity penalty is visible.
- **23 issues** spread across beginner / intermediate / advanced, with realistic bodies, reproduction steps, labels and comment threads. Several are designed to be *rejected* by the hard filters (security-labelled, already assigned) so the funnel is demonstrable rather than asserted.
- **3 contributor profiles** with hand-built skill graphs and evidence strings: `alex` (backend Python/FastAPI, never contributed), `priya` (frontend TypeScript/React/a11y, some OSS), `rahu1` (infrastructure Go/Kubernetes, experienced).
- **41-skill taxonomy** across language, framework, infrastructure, data, engineering and AI categories.

Live mode is the same code path: sign in with GitHub, and `?refresh=true` pulls real issues through the same filters and scorer.

7. Running it

```
# Backend — http://localhost:8000 (seeds itself on first boot)
cd backend
python -m venv .venv && .venv/Scripts/activate      # Windows
pip install -r requirements.txt
uvicorn app.main:app --reload

# Frontend — http://localhost:3000
cd frontend
npm install
npm run dev

# Tests
cd backend && pytest -q          # 43 passed
```

All configuration is optional. With no `.env` at all the demo runs end to end on heuristics.

Variable	Effect when set
<code>GROQ_API_KEY</code>	Enables LLM enrichment; absent → deterministic mode
<code>GITHUB_CLIENT_ID</code> / <code>_SECRET</code>	Enables real GitHub sign-in
<code>GITHUB_TOKEN</code>	Raises unauthenticated discovery from 60/hr to 5000/hr
<code>DATABASE_URL</code>	Postgres instead of SQLite
<code>INGEST_TOKEN</code>	Enables the batch ingest endpoint; unset disables it entirely rather than leaving it writable

8. Measurable success metrics

The headline product metric, and the one worth instrumenting first:

Recommendation-to-first-PR conversion — of issues a user opens the workspace for, the share that reach an opened pull request within 14 days.

Baseline to beat: browsing labelled issue lists unaided, where the overwhelming majority of opened issues are abandoned before any code is written.

Supporting metrics, each cheap to collect:

Metric	Definition	Why it matters
Plan step completion	Share of plan steps a user marks done	Detects exactly which step loses people (we expect step 4, dependency install)
Effort calibration error	predicted hours – actual time to PR	Directly falsifiable — the estimate is a claim, so measure it
Filter precision	Share of surfaced issues later found assigned/stale/closed	Measures the hard filters, the one stage with no model in it
Grounding violations	Plan or chat responses naming a file absent from the repository tree	Should be zero; automatically checkable against <code>repo.tree</code>
Time to first candidate	Sign-in → first ranked recommendation	Currently sub-second on the seeded corpus

Grounding violations are the sharpest of these: the check is mechanical, the target is zero, and it tests the exact property that makes a naive LLM unusable here.

9. Known limitations

Stated plainly, because pretending otherwise would undercut the rest of the document.

1. **Affected-file inference is token overlap, not program analysis.** It scores repository paths against issue vocabulary. It is labelled an inference in the UI and is frequently useful, but it is a heuristic — a real AST or call-graph analysis would need the repository cloned, which the prototype does not do.
2. **Skill confidence is not measured proficiency.** It is a heuristic over repository languages, manifests and merged PRs. A developer with private-only work will be scored low on evidence they genuinely have. Self-reporting can raise a skill but never silently demotes one.
3. **Retrieval is lexical.** TF-IDF misses conceptual matches that share no vocabulary. Adequate at a few hundred candidates; embeddings become necessary as the corpus grows.

4. **Effort estimates are difficulty-bucket ranges**, not learned from outcome data. No completion data exists yet — metric 2 above is designed to fix this.
 5. **GitHub rate limits bound live discovery**. Unauthenticated search is 60/hr. The limit surfaces to the UI as a countdown rather than a silent failure, but it is real.
 6. **The seeded corpus is synthetic**. It is realistic and deliberately varied, but it is not a random sample of GitHub, so ranking quality on it is illustrative rather than an evaluation result.
 7. **No human-in-the-loop write actions**. By design for this prototype — the agent never comments, claims or files on the user's behalf. That is the correct default, but it does mean the last mile is manual.
-

10. Roadmap

Near term - Outcome tracking: watch for the PR that references the issue, close the loop on effort calibration, and train the estimate on real completion data. - Grounding-violation CI check: assert that every file path in a generated plan exists in the repository tree. Mechanical, and it protects the core property. - Plan step completion tracking, to find the step where contributors actually stall.

Medium term - Embedding retrieval behind the same interface, once corpus size justifies it — the TF-IDF index is a swappable component with a deliberately narrow contract. - Maintainer-side view: which issues are attracting qualified contributors and which are silently repelling them. - Repository-specific onboarding built from merged-PR history rather than generic first-contribution steps.

Scale and deployment - Stateless API; SQLite → Postgres is a connection-string change. Issue ingestion already runs out-of-process through `POST /api/ingest/issues` behind a shared secret, so corpus refresh scales on a scheduler independent of the app. - Analysis is cached per issue and reused across all users — cost per issue is paid once, not per contributor, so marginal cost per user approaches the retrieval cost alone.

11. Evaluation-dimension map

Dimension	Where to look
Problem relevance and clarity	§1 — a specific user, a measured drop-off point, and four named alternatives with the gap stated
Quality and meaningfulness of AI implementation	§5 — a per-sub-problem argument for where AI is and is not used, a four-stage pipeline confining the model to prose, and an enforced fallback contract
Technical depth and execution	§4 — tool layer, extraction, ranking, 43 tests, encrypted tokens, rate-limit handling
Innovation and originality	Grounded contribution <i>plans</i> rather than issue summaries; reproducible scoring that shows its work; correct behaviour with no model available
User experience and usability	Provenance and confidence on every AI surface; the filter funnel reported honestly; "what the agent does not know" as a first-class panel
Practical feasibility and scalability	§10 — stateless API, out-of-process ingestion, per-issue analysis cache shared across users
Demonstrated impact	§8 — conversion to first PR, with falsifiable supporting metrics
Quality of final demo	Runs with no API key, no GitHub token and no network on a seeded corpus